

CLONE 3.0: UI Panel Redesign and Improvements

Amaro Da Luz
adaluz@stevens.edu
Stevens Institute of Technology
Hoboken, New Jersey, USA

Fintan Naughton
fnaughto@stevens.edu
Stevens Institute of Technology
Hoboken, New Jersey, USA

Jake Stinton
jstinton@stevens.edu
Stevens Institute of Technology
Hoboken, New Jersey, USA

Juliana Pimentel
jpimente@stevens.edu
Stevens Institute of Technology
Hoboken, New Jersey, USA

Dhir Rana
drana5@stevens.edu
Stevens Institute of Technology
Hoboken, New Jersey, USA

ABSTRACT

CLONE is an IntelliJ IDEA plugin, implemented to detect and refactor duplicate code interactively as soon as a duplicate is introduced. The plugin only recommends the extraction of a duplicate when it is worth it. The goal of this research project was to create a UI panel for the user. This work extends our tool to allow developers to easily spot duplicated code, compare the suggested refactoring with the original code, and edit, apply, or reject it. We also introduced an AI Validation Score to increase trust between the user and the plugin. The source code is publicly available on GitHub at <https://github.com/refactorings/anti-copy-paster>. The demonstration video can be found on YouTube: (COMING SOON) [1]

CCS CONCEPTS

• **Software Engineering** → **Software Testing, Quality Assurance, and Maintenance**;

KEYWORDS

software quality, mining software repositories, refactoring, duplicated code, software quality

1 INTRODUCTION

When developing software, it is common for the same logic to be run in numerous different segments of code. For example, the software might be searching for an input, such as an existing user and matching password in one section, or a title for a project in a different section. The most frequently used solution by developers is to copy and paste the already existing logic, and simply change a variable or two. However, one major issue is the possibility that the pasted code will later need to be tweaked. If something is fundamentally wrong with the code, or if it could be optimized in any form, the developer would need to change every single pasted section of the code. If any one were to be forgotten, it could introduce inconsistencies that lead to bugs, and the duplicated code adds unnecessary bulk to the codebase. So, how should developers prevent this possibility from occurring?

An existing and popular solution is called Extract Method refactoring, which involves moving a cohesive fragment of code into a separate, clearly named method. This means that those multiple pasted fragments will become method calls instead, and tweaking the pasted code means only one segment will need to be changed. However, there is one central issue with this method: the developer must notice the clone themselves and decide to refactor. Depending on

how late the developer decides to refactor, there may be numerous pasted segments, and developers may overlook one or more segments in their attempt to refactor. To address this, some developers may use clone-detection tools to analyze their code and ensure all pasted fragments are refactored. However, they are often used after the code has been written and are not used in the moment. But what if detection happened at the moment of pasting, so refactoring is suggested immediately instead of discovered later on?

This is where CLONE, an IntelliJ plugin that extends real-time clone detection with machine learning, comes into play. CLONE is built on top of AntiCopyPaster, a plugin that monitors paste events inside the IDE and, upon detecting a duplicate, decides whether the fragment is worth extracting. Rather than requiring the developer to search the codebase after the fact, AntiCopyPaster frames this as a binary classification problem: it computes a broad set of structural and semantic metrics from the pasted fragment, covering categories such as keyword usage, coupling, complexity, and size, and feeds them into a convolutional neural network trained on a large dataset of real-world Extract Method refactorings [1]. If the fragment is flagged, the plugin suggests the refactoring and applies it immediately, all within the developer's existing workflow. One limitation reported by users of this approach, however, is its black-box nature. The CNN's decision offered no visibility into why a fragment was or wasn't flagged, and developers had no way to influence that decision. A later version of the plugin addressed this at the detection stage by letting developers configure and weight the metrics themselves [1]. CLONE builds on this same real-time detection foundation, but adds the option to hand off detection and refactoring to an LLM-based agent (Aider) instead. Rather than analyzing only the pasted fragment, this mode can scan a single file or an entire set of files for clones, walking the developer through each step: notifying them that analysis is running, flagging any clones found, and presenting the proposed refactoring in a diff viewer for review before applying it.

While CLONE laid the groundwork for real-time clone detection and refactoring, this development cycle introduces several improvements to make the tool more usable and reliable for developers. In particular, while prior work addressed the opacity of the detection decision by letting developers configure metrics themselves, the refactoring recommendation itself still gave no indication of how confident the tool was in a given suggestion; a gap this paper's usefulness rating directly addresses. This paper details these and other contributions:

- We redesigned the pop-up that appears when multiple clones are detected, allowing developers to review and choose among candidate matches rather than acting on a single one
- We introduced a usefulness rating reflecting CLONE’s confidence in a given refactoring. Surfacing this score gives developers a quick signal for how trustworthy a given suggestion is before they commit to applying it, rather than treating every suggestion as equally reliable.
- We redesigned the pop-up that appears when multiple clones are detected, simplifying its layout and formatting so developers can more easily understand and act on each candidate occurrence as they step through them.

The remainder of this paper is organized as follows:

- Problem Statement
- Workflow
- How to Use
- New Features
 - Multiple Clone Dialog
 - Explanation Section
 - Diff View
 - AI Trust Panel
 - Action Buttons
- Experimental Results
- Conclusion
- References

2 PROBLEM STATEMENT

Our research project addresses the problem of duplicated Java code caused by copy-and-paste programming. Copying and pasting code can help developers work quickly, but it can also create repeated logic across the code, making it harder to maintain and update over time. Although the previous version of CLONE could detect duplicate code and generate refactoring suggestions, the original interface didn’t present the result in a way that is clear, easy to understand, and trustworthy for developers to act on.

Users struggled to understand exactly where the duplicated code was, why it was classified as such, what the proposed changes were, and to what extent they should trust the proposed refactoring. This is important because developers should not have to accept an AI-generated refactoring without understanding how the tool reached that suggestion or whether that change is safe.

Our project therefore focuses on redesigning and implementing a clearer IntelliJ-style user interface for CLONE. The improved UI explains code details, supports multiple clone occurrences, shows before-and-after code, displays AI Trust and validation information, and gives the developer total control over whether to apply, edit, or reject the suggestions. The goal is to make CLONE more understandable, transparent, reliable, and user-friendly for Java developers while helping them reduce duplicated code safely.

3 WORKFLOW

CLONE is designed to function as an IntelliJ plugin that seamlessly fits into the already present programming workflow. It is capable of detecting clones in when code is pasted in, automatically

refactoring clones for the user where it is deemed useful, while also maintaining the original program functionality.

The user is given a large amount of control throughout the refactoring process, in order to ensure that unwanted changes are not made to the program. The user is able to individually select found clones to determine if they wish for those clones to be refactored and is able to alter the refactoring made, either by re-prompting the AI model or by manually editing the refactoring themselves.

CLONE detects redundancy from copy and pasted methods by using PSI trees to judge the overall behavior of the program. The user also has the ability to select the method of naming a refactored method, either from IntelliJ’s pre-made name recommendation system or by using the Code2Vec neural model.

There are five major steps in CLONE’s process, being the Detection Agent, Refactoring Agent, Usefulness Agent, Compilation Agent, and Testing Agent. The first step in the process is the Detection Agent, which looks throughout the project to find segments of code that match up with the segment of code that was just pasted in. It also attempts to classify matching segments into clone types, which becomes helpful when refactoring.

If the Detection Agent successfully locates a clone, then the Refactoring Agent begins to create a suggested refactoring of the code. It will create a new method in the program that can be called to achieve the same functionality when placed in where the clones once were. Once a refactoring has successfully been created, the Usefulness Agent ensures that the refactoring that has been created actually accomplishes something helpful, rather than, for example, creating a refactoring that does not make any calls to the new method created.

Following that, the Compilation Agent simply checks if the refactoring suggestion still compiles when implemented into the project. This ensures that there are no major errors structurally within the code. Finally, the Testing Agent ensures sure that the refactored code maintains the same functionality as the original code by running tests on both the original and updated versions of the code to make sure that outputs from the code will match. Once the Testing Agent has completed its process, the user will be shown the suggested refactoring and be provided with the option to implement, ignore, or edit the refactoring.

If at any point throughout the process a failure occurs at one of these five steps, the entire process will iterate up to a certain amount of times specified by the user. When a failure occurs at a step after the Refactoring Agent, feedback is even provided to the Refactoring Agent to help it improve on later iterations.

4 HOW TO USE

CLONE, originally developed under the name AntiCopyPaster, is distributed as a plugin for IntelliJ IDEA.¹ The plugin can be downloaded as a ZIP file from its GitHub repository.

¹<https://github.com/refactorings/anti-copy-paster>

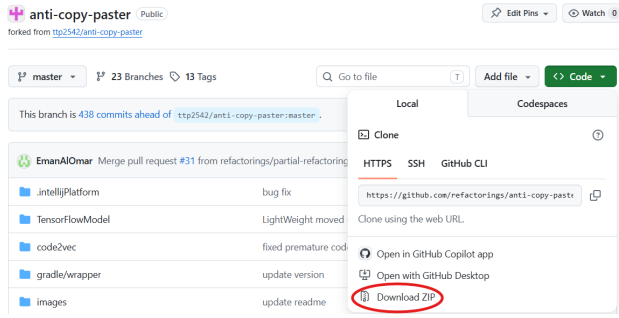


Figure 1: Install CLONE plugin to ZIP

Once downloaded, the plugin can be installed directly within IntelliJ IDEA. After opening the target project, the developer navigates to **Settings**, then to the **Plugins** section.

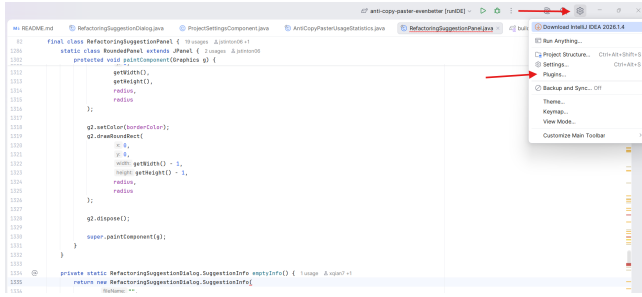


Figure 2: Accessing Plugin settings in IntelliJ

From there, clicking the gear icon within the Plugins panel and selecting **Install Plugin from Disk** allows the developer to select the downloaded ZIP file. Once selected, the IDE must be restarted to complete installation.

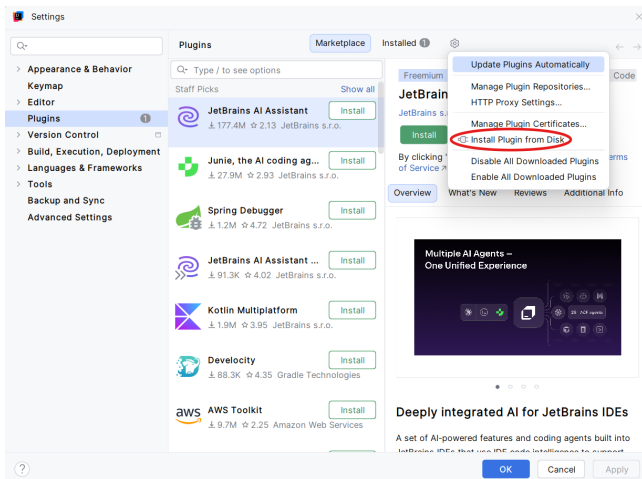


Figure 3: Installing CLONE plugin within IntelliJ

After restarting, the developer returns to **Settings** and navigates to **Tools**, where the newly installed CLONE plugin can be configured. Under the option labeled **Extraction judgements should be performed by**, the developer selects **Clone_multiagent**, restarting the IDE again if prompted. The developer then selects a supported AI model provider, such as Google Gemini or Anthropic Claude, and enters the corresponding information under **Model Suites** and **Model Selection**. An API key for the selected provider must then be obtained and entered into the appropriate field; each supported provider makes its API keys available through its own developer console (e.g., Google AI Studio for Gemini, or the Anthropic Console for Claude).

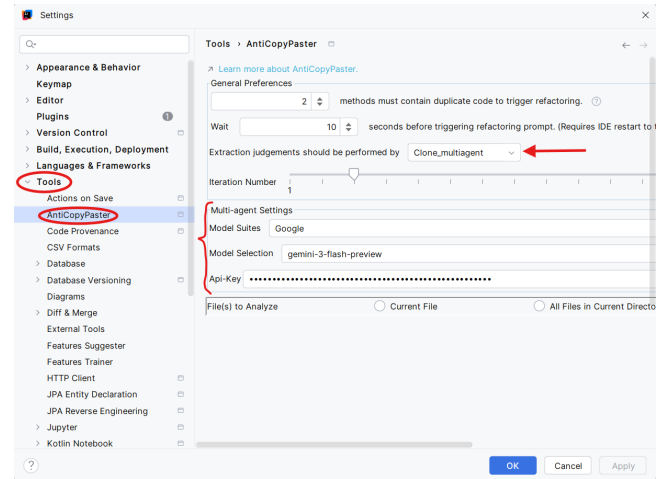


Figure 4: Proper settings for CLONE

Once this configuration is complete, CLONE is ready for use. From this point forward, any copy-and-paste action performed within the IDE will automatically trigger Clone_multiagent's detection process. If the pasted fragment is found to introduce one or more clones, CLONE begins its refactoring workflow, guiding the developer through the detection, review, and refactoring steps described in the preceding sections.

5 NEW FEATURES

The updated CLONE interface introduces several changes to improve the clarity, inspection, and regulation of AI-based refactoring suggestions. To achieve these goals, the newly added features are expected not only to highlight the presence of a refactoring suggestion, but also to justify the reasoning behind its emergence, to reveal the differences between the generated code and the code duplicate used to generate it, and to ensure that the suggested refactoring is safe enough to be used by the programmer.

Listed below are the main objectives pursued in the redesigned CLONE interface:

- The developer needs to have the option to see several instances of clones detected to determine which ones can be included in the refactoring process.

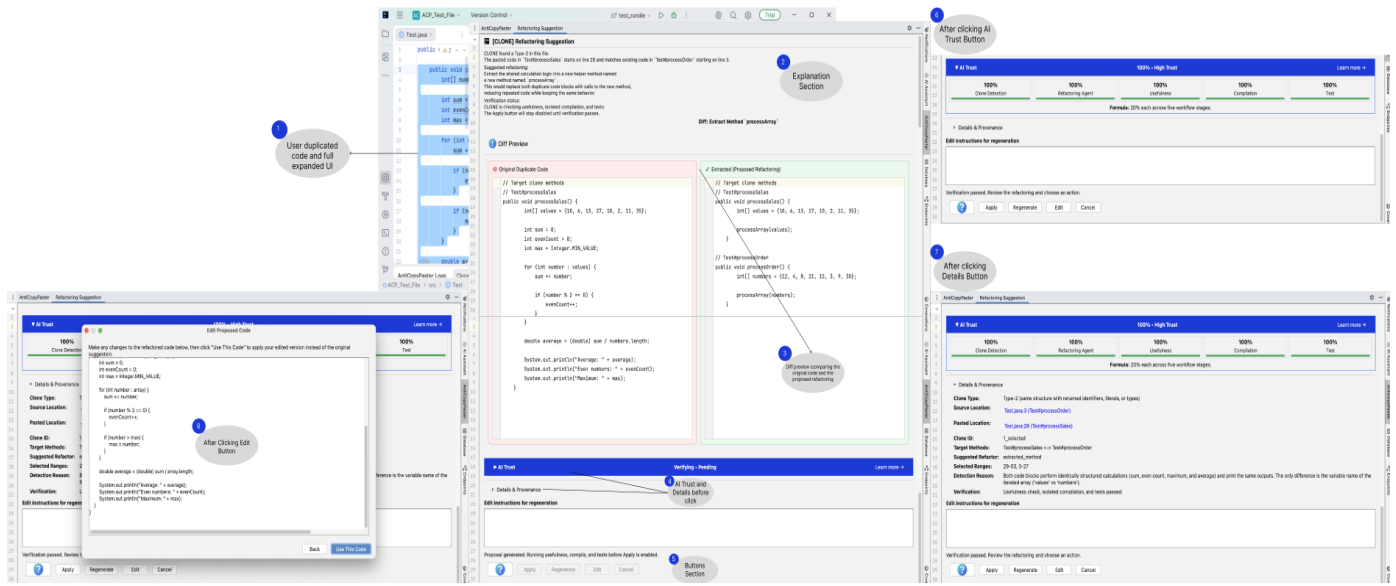


Figure 5: Full UI Panel

- The developer needs to have a brief explanation about the detected clone, where it is located, and how the recommended refactoring (Extract Method) should work.
- The developers need to be able to compare the source code of duplication with the refactored version in the improved Diff View.
- The developers need to have the ability to see how reliable the suggestion made by the AI is using an AI Trust panel showing the validation results.
- The developers need to keep the right to make the final decision using the action buttons.

Next, we describe each of these features and how they improve the usability, transparency, and reliability of the CLONE refactoring workflow.

5.1 Multiple Clone Dialog

When CLONE detects that a pasted fragment matches multiple existing occurrences in the codebase, it opens a dialog that steps the developer through each candidate one at a time. The dialog reports how many total occurrences were found and, for each one, asks the developer to include or exclude it from the refactoring before advancing to the next. For example, if CLONE detects three occurrences, the developer would be asked in turn about "Occurrence 1/3," "Occurrence 2/3," and "Occurrence 3/3." We redesigned this dialog to simplify its layout and formatting, making it easier for developers to understand which occurrence they're looking at and what action they're being asked to take at each step. This step takes place before CLONE attempts any refactoring, ensuring that only the occurrences the developer has explicitly included are modified. The redesign followed a set of Figma mockups established prior to implementation, ensuring the interface matched an intended visual and interaction design. Technically, the dialog was implemented as a CloneOccurrenceDialog, built using IntelliJ Platform's

DialogWrapper pattern, which integrates natively with the IDE's UI conventions. Call sites throughout the existing refactoring workflow were updated to invoke this new dialog in place of the previous implementation. With this redesign, developers can move through each detected occurrence with a clear sense of what they're looking at and what decision is being asked of them, reducing the chance of confusion or accidental inclusion/exclusion during the refactoring process.

5.2 Explanation Section

The developer needs to quickly understand what was detected, where it is located, and what Extract Method refactoring is being suggested. That is the objective of the Explanation Section. It is the first section of the UI Panel and is useful because it provides users with essential information immediately after the duplicated code is detected. We concluded that the information it provides had to be concise and with not much detail, for it to be user-friendly while providing the most crucial information the users needs.

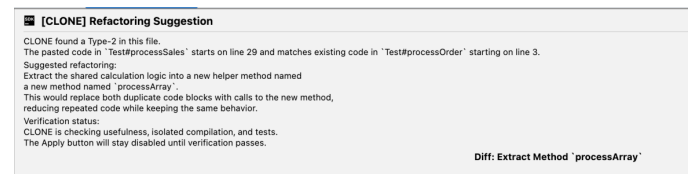


Figure 6: Explanation Section

5.3 Diff View

When the refactored code is generated, it is shown and compared with the original duplicated code in the Diff View panel. This side-by-side comparison is done by the two rounded cards displaying IntelliJ code viewers. The first one, in red, shows the original snippet

of duplicated code, and the second, in green, shows the refactored code. The original code viewer was a single, draggable panel divided in two, taking up much of the user’s view and displaying the code in an awkward, messy layout. The header was also altered with the intention of de-cluttering the entire Diff View sub-section, originally polluted with unproductive buttons and replaced with a large information icon calling for the user’s attention.

These alterations allowed for a new Diff Panel that better communicates the exact changes made by CLONE. The colors and icons added bring the user’s attention to the refactoring suggestion, and the two separated editors make analyzing the code changes a less confusing and cluttered experience. With this, the user is able to identify where the code duplicate can be found within their file and carefully evaluate the refactoring suggestion generated by the plugin. From there, the user can decide to include, reject, or alter the refactored code through buttons found in a separate section of the CLONE panel.

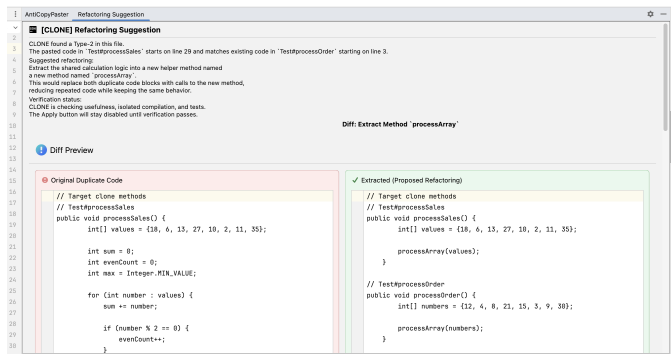


Figure 7: Diff view showing the original duplicated code alongside the extracted proposed refactoring.

5.4 AI Trust Panel

The AI Trust panel is one of the innovations implemented in the new version of the CLONE refactoring process. The idea behind this tool is to increase the transparency of AI-generated refactoring suggestions before they are applied. In the previous version of the tool, the interface could make refactoring suggestions, but it did not provide much information about the reliability of those suggestions or the validations that supported them. Since applying a refactoring means modifying the source code, developers need validation information before deciding whether to accept the change. The AI Trust panel addresses this by placing validation information directly within the Refactoring Suggestion panel.

The panel measures the trustworthiness of a suggested refactoring through five validation criteria: clone detection, refactoring agent confidence, usefulness of refactoring, compilation, and test/behavior preservation. Each criterion contributes 20% to the total trust value. We apply equal weighting because each of these steps represents a different phase of the refactoring process. Clone detection measures the correctness of duplicate code identification; refactoring agent confidence is related to the accuracy of the produced transformation; usefulness measures the relevance of the

refactoring suggestion; compilation determines whether the refactoring builds successfully; and test/behavior preservation checks whether the transformation preserves the program behavior.

This is significant because there are several ways for an AI-based refactoring to fail. It may correctly detect duplicated code, but the generated transformation may not compile. Likewise, even if the code compiles successfully, the transformation may still be less useful if the extracted method is confusing or unnecessary to the programmer. By breaking down the trust score into different validation scores, the interface avoids presenting trust as one unexplained overall number. Instead, it makes clear which part of the process influenced the final recommendation.

The AI Trust panel also provides different states for validation. If validation is successful, the panel shows the trust score, trust level, and each component score. If the validation process has not been completed yet, the panel indicates a pending state, helping the developer understand that the recommendation has not been fully validated. If validation fails, the panel provides unavailable values rather than unsupported confidence levels. This prevents the developer from getting a false sense of certainty in cases where one or more validations were unsuccessful.

This design makes sure that the panel is compact and collapsible so that it does not take too much space away from the explanation pane, diff preview, or action buttons. This is crucial since programmers need to evaluate both the proposed code changes and the validation results at the same time. The placement of the AI Trust panel close to the diff preview creates a connection between the refactoring output and its validation evidence. As such, this feature helps make CLONE more interpretable and trustworthy.

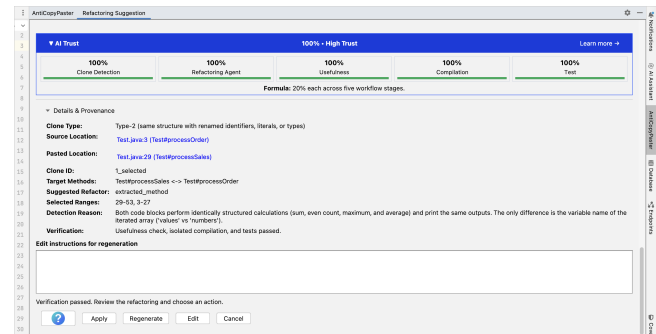


Figure 8: AI Trust panel showing the overall trust score, five validation components, and provenance details for the selected refactoring recommendation.

5.5 Action Buttons

The updated Refactoring Suggestion panel gives developers greater control over AI-generated recommendations through improved action buttons. A new *Edit* button allows the user to manually revise the proposed refactoring before applying it to the project. This is useful when a recommendation is mostly correct but requires small project-specific adjustments.

The behavior of the *Apply* button was also changed to improve safety. The button remains disabled while validation is still running or when the refactoring workflow fails. This provides an extra layer

of protection to ensure unfinished or unsafe code cannot be applied to the source file.

Together, these actions support a human-in-the-loop workflow in which developers can accept, revise, regenerate, or dismiss an AI-generated recommendation.

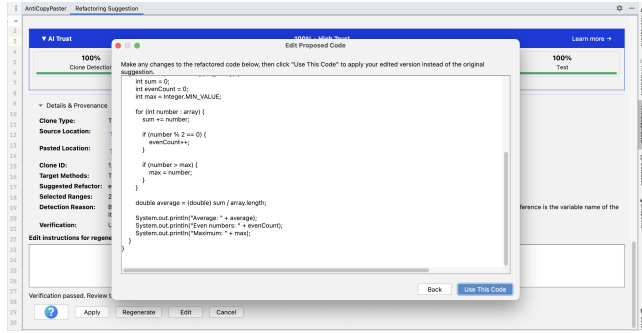


Figure 9: Edit workflow allowing the developer to revise the generated refactoring before applying the updated code to the project.

6 EXPERIMENTAL RESULTS

To evaluate the new CLONE interface, we collected feedback from eight users after they tested the plugin. We focused on whether the users could understand the detected clone, review the proposed refactoring, and decide whether to apply, edit, or reject the suggestion.

Overall, the results were positive. As shown in Figure ??, 62.5% of users rated their overall experience with the CLONE UI Panel as "Good", while the remaining 37.5% rated it as "Neutral". No participants rated it as a negative experience. This suggests that all users found the interface acceptable while also indicating that it could still have room for improvement.

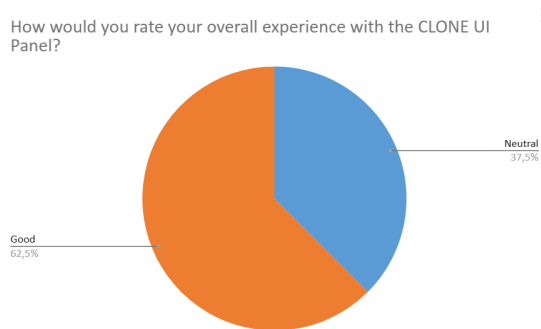


Figure 10: Overall user experience rating for CLONE UI Panel

The usefulness results are shown in Figure 11. The Diff Preview was selected as the most useful feature of the UI Panel. These results suggest that users valued features that made the proposed refactoring more reviewable and transparent.

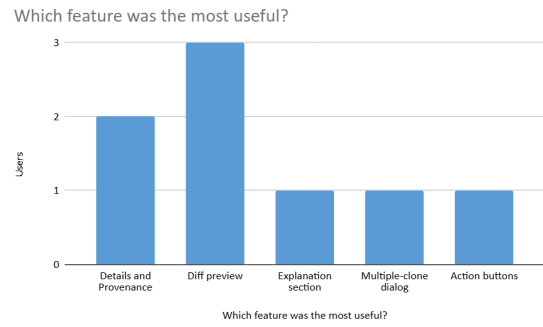


Figure 11: User responses for the most useful CLONE UI feature

This shows the importance of reviewability and developer control. The Diff Preview received one of the strongest ratings, with the users agreeing that it clearly showed the difference between the original code and the proposed refactoring. Users also reported that the panel provided enough information to make a decision that they would feel comfortable applying. These results support the goal of presenting AI-assisted refactoring as a transparent workflow rather than as an unexplained suggestion.

However, the feedback also revealed several limitations. Some users reported technical issues such as API key problems, model/provider errors, setup difficulties, or cases where the flow did not complete completely. In addition, some users reported that the AI Trust and validation information, multiple-clone dialog, or action buttons could be clearer. These responses show that while the interface improved the review process, future work should focus on making the setup easier, clarifying validation states, improving layout readability, and making failure states easier to understand.

Overall, the user feedback suggests that CLONE's interface helps developers better understand and review duplicate code refactoring suggestions. The most useful feature was the Diff Preview, which supports the goal of making the proposed code changes visible before they are applied. At the same time, the neutral overall ratings and reported technical issues indicate that the tool would benefit from further usability improvements and a better handling of model, token, and configuration errors.

7 CONCLUSION

This paper introduces the UI changes made to CLONE and evaluates whether they made the IntelliJ plugin more user-friendly. Throughout the project, we updated the pop-up dialog that appears when CLONE detects multiple clone occurrences, along with the buttons users interact with, including the addition of a more useful help button. Additionally, we redesigned the explanation describing why a refactoring was suggested, along with the preview window showing the suggested code changes. Finally, we introduced an AI confidence score meant to indicate how effective the tool believes a given refactoring would be.

These changes were well received by users overall, with 63% reporting a "good" experience with the tool. The majority of testers found the new features easy to use and understand, and only one tester did not find the AI Trust score or the edit option useful.

Additionally, all testers indicated that they would trust a refactoring suggested by CLONE, and that the redesigned UI made the plugin more understandable.

Despite this positive reception, the tool is not without its limitations: 63% of users also reported technical issues installing or using the tool. Because responses were collected anonymously, it is not possible to determine whether this reflects the same group of users who rated their experience positively. These issues included an overly complicated installation process, the refactoring threshold failing to be met even when sensitivity was set to zero, refactoring suggestions introducing redundancy into the code, and some UI elements still displaying the older "AntiCopyPaster" title.

Overall, our findings show that these efforts have made CLONE more user-friendly. The updated features were found to be easy to understand and use, and the majority of testers indicated they would consider using the tool in future Java projects. Future work will focus on resolving the installation and detection issues identified during testing to further improve the tool's reliability.

REFERENCES

- [1] Eman Abdullah AlOmar, Jacob Ashkenas, Robert Feliciano, Matthew Angelakos, Dimitrios Haralampopoulos, Xing Qian, Mohamed Wiem Mkaouer, and Ali Ouni. 2026. Anticopypaster 3.0: Just-in-time clone refactoring. *ACM Transactions on Software Engineering and Methodology* 35, 5 (2026), 1–52.